Interrogation Avancée en SQL : Jointures Relationnelles et Regroupements (GROUP BY & HAVING)

Thème principal : Bases de données et langage SQL avancé

Synthèse exécutive : Ce cours de la ressource R1.05 aborde les mécanismes fondamentaux du langage SQL pour croiser des données issues de multiples tables relationnelles via les jointures (INNER JOIN, LEFT JOIN) et structurer les résultats par groupes au moyen des clauses GROUP BY et HAVING. L'objectif est d'assimiler la logique d'exécution des requêtes complexes, de maîtriser le calcul d'agrégats statistiques (AVG, COUNT, SUM) et d'éviter les pièges classiques tels que la génération involontaire de produits cartésiens ou l'utilisation erronée de fonctions d'agrégation dans la clause WHERE.

Les Jointures Relationnelles : INNER JOIN vs LEFT JOIN et prévention du Produit Cartésien

Dans un SGBDR (Système de Gestion de Base de Données Relationnel), la normalisation impose la dispersion contrôlée des données entre plusieurs tables. La jointure est l'opération algébrique indispensable pour réassembler ces informations au moment de l'interrogation. La clause JOIN est évaluée au tout début de l'exécution dans le bloc FROM. Il est obligatoire de spécifier le prédicat de liaison (ex: ON E.id_etud = N.id_etud) reliant la clé étrangère d'une table à la clé primaire de l'autre table. Sans cette clause ON, le moteur SQL applique un produit cartésien, combinant chaque enregistrement de la première table avec l'ensemble des enregistrements de la seconde table, risquant d'entraîner le plantage ou le surcoût mémoire du serveur. L'INNER JOIN filtre le résultat en ne conservant que les correspondances effectives, tandis que le LEFT JOIN garantit que les enregistrements de la table de gauche apparaissent tous dans le résultat final, en injectant la valeur NULL pour les attributs de droite en l'absence de correspondance.

- Normalisation relationnelle : Séparation des entités métiers dans des tables distinctes reliées par Clé Primaire (PK) et Clé Étrangère (FK) pour éliminer les redondances.
- INNER JOIN (Jointure Interne) : Opération d'intersection restituant exclusivement les paires de n-uplets satisfaisant la condition d'équivalence déclarée dans la clause ON.
- LEFT JOIN (Jointure Externe Gauche) : Conservation de l'exhaustivité de la table de gauche, complétée par des valeurs NULL si aucune correspondance n'existe dans la table de droite.
- Produit Cartésien (CROSS JOIN) : Évaluation catastrophique causée par l'omission du critère de jointure (clause ON), générant un croisement combinatoire N x M exhaustif et très coûteux en performances.

Cas concret enseignant : *Le professeur compare l'INNER JOIN à l'accès sécurisé par portique au campus de l'IUT de Bayonne : seuls les étudiants possédant une carte valide ET une inscription effective peuvent franchir la porte. Si une correspondance manque d'un côté, la ligne est éliminée. Pour le LEFT JOIN, l'analogie est la liste d'appel globale du secrétariat de l'IUT : tous les étudiants inscrits apparaissent obligatoirement sur le registre ; si un étudiant n'a pas encore de note saisie, son bulletin affiche la valeur 'NULL'.*

Regroupement de Données et Agrégation Statistiques : Clauses GROUP BY et HAVING

L'agrégation permet de réaliser des synthèses décisionnelles sur des données. Dès que la clause GROUP BY est invoquée, la table initiale est scindée en partitions indépendantes. Une erreur conceptuelle fréquente en partiel consiste à essayer de filtrer un calcul statistique dans la clause WHERE (par exemple WHERE AVG(note) > 10). Cela est techniquement impossible : la clause WHERE s'exécute sur les lignes individuelles de la table source AVANT que les groupes ne soient constitués et que les fonctions agrégées ne soient calculées. SQL fournit la clause HAVING pour appliquer des critères de restriction postérieurs au regroupement. De plus, il est formellement interdit d'insérer dans la clause SELECT un attribut non agrégé s'il n'est pas déclaré dans le GROUP BY, sous

peine d'erreur sémantique SQL.

• Fonctions d'agrégation (COUNT, SUM, AVG, MIN, MAX) : Opérations statistiques réduisant un ensemble de n-uplets à une valeur scalaire unique.

• Clause GROUP BY : Partitionnement de la table filtrée en sous-groupes homogènes partageant la même valeur sur les attributs spécifiés.

• Règle d'or du SELECT avec GROUP BY : Toute colonne présente dans le SELECT doit obligatoirement figurer dans le GROUP BY ou être enveloppée dans une fonction d'agrégation.

• Clause HAVING : Prédicat de filtrage s'appliquant exclusivement après le regroupement sur les sous-groupes formés et les résultats agrégés.

Cas concret enseignant : *L'enseignant illustre le fonctionnement avec le dépouillement physique des copies d'examen dans l'amphithéâtre de l'IUT : 1. GROUP BY : Le professeur répartit physiquement toutes les copies dans trois bannettes distinctes selon le département (Informatique, GEA, TC). 2. Agrégation (AVG) : Pour chaque bannette, il calcule la moyenne générale. 3. HAVING : Si le directeur exige de ne conserver que les paquets dont la moyenne dépasse 12/20, c'est la clause HAVING qui élimine les bannettes non conformes. La clause WHERE aurait servi avant le tri à écarter les copies des étudiants fraudeurs.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi est-il strictement impossible d'écrire 'WHERE AVG(note) > 10' dans une requête SQL ?
Réponse du professeur : Parce que la clause WHERE est évaluée ligne par ligne au niveau de la table brute avant même que le regroupement (GROUP BY) ne soit effectué et que les fonctions d'agrégation ne soient calculées.

Question étudiant : Que se passe-t-il si j'oublie d'inclure dans la clause GROUP BY une colonne présente dans le SELECT ?
Réponse du professeur : Le SGBD renvoie une erreur sémantique, car il ne sait pas quelle valeur afficher pour cette colonne parmi toutes celles contenues dans le groupe formé.

■ Exercice & Démonstration d'Application

Énoncé : Soit la base de données relationnelle de l'IUT composée des deux tables : Etudiant(id_etud, nom, id_promo) et Note(id_note, id_etud, matiere, note). Écrire la requête SQL permettant d'afficher le nom de chaque étudiant et sa moyenne dans la matière 'R1.05', uniquement pour les étudiants possédant au moins 2 notes enregistrées dans cette matière et une moyenne stricte supérieure à 12/20. Le résultat doit être trié par moyenne décroissante.

Démarche de résolution :
1. Assemblage des tables (FROM / JOIN) : On réalise une équijointure interne entre Etudiant (alias E) et Note (alias N) sur le champ pivot id_etud. 2. Filtrage des lignes individuelles (WHERE) : On restreint les notes consultées à la matière 'R1.05' avant tout calcul. 3. Partitionnement en groupes (GROUP BY) : On regroupe les enregistrements par étudiant (id_etud et nom). 4. Filtrage des groupes (HAVING) : On exprime les deux conditions agrégées : compter au moins 2 notes (COUNT >= 2) et une moyenne > 12. 5. Projection des attributs (SELECT) : On extrait le nom et la moyenne calculée renommée. 6. Tri des résultats (ORDER BY) : On trie le jeu final par ordre décroissant de la moyenne.

```
SELECT E.nom, AVG(N.note) AS moyenne
FROM Etudiant E
INNER JOIN Note N ON E.id_etud = N.id_etud
WHERE N.matiere = 'R1.05'
GROUP BY E.id_etud, E.nom
HAVING COUNT(N.id_note) >= 2 AND AVG(N.note) > 12
ORDER BY moyenne DESC;
```

Résultat : La requête renvoie un tableau contenant le nom des étudiants qualifiés et leur moyenne calculée en R1.05, triés du meilleur au moins bon.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'application entre la clause WHERE et la clause HAVING en SQL ?

R : La clause WHERE filtre les lignes individuelles avant le regroupement, tandis que la clause HAVING filtre les groupes formés après le calcul des fonctions d'agrégation.

Q : Quel est le résultat produit par une requête avec LEFT JOIN si un enregistrement de la table de gauche ne possède aucune correspondance dans la table de droite ?

R : L'enregistrement de gauche est conservé dans le résultat et les colonnes issues de la table de droite sont complétées par la valeur NULL.

Q : Si la table 'Etudiant' contient 10 enregistrements et la table 'Note' contient 5 enregistrements, combien de lignes retourne un produit cartésien (CROSS JOIN) entre ces deux tables ?

R : Il retourne 50 lignes (10 x 5), correspondant au croisement exhaustif de chaque ligne de la première table avec toutes les lignes de la seconde.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765